LEVEL 5: CI/CD

★ Push-to-Production Pipeline for shop-api

Level 5 capstone ~12 hours

1 Goal and outcome

Build the roadmap's practice pipeline end to end for the `shop-api` sample application:

Git Push → GitHub → Lint → Tests → Build → Docker Build → Security Scan → Push Image → Deploy → Health Check → Notify, with an automatic **rollback** when the health gate fails.

When you finish, a `git push` to `main` will: lint the code, the Dockerfile and the workflows; run unit and integration tests against real PostgreSQL and Redis containers; build the image with a layer cache; scan it and fail on fixable HIGH or CRITICAL vulnerabilities; push it by digest to GHCR with signed provenance; deploy the same digest to **staging** and then (after approval) **production** on a Linux server using a **blue/green** switch with a health gate and bake time; roll back by itself if the new version misbehaves; and post the result to Slack. You will also be able to roll back with one button, and you will have rehearsed the failure cases.

This ties together every topic of the level: L5.1 pipeline design, L5.2 GitHub Actions, L5.3 testing, L5.4 Docker builds and registries, L5.5 secrets, L5.6 artifacts, L5.7 rollback, L5.8 zero-downtime, L5.9 migrations, L5.10 rolling, L5.11 blue/green and L5.12 canary.

2 Architecture

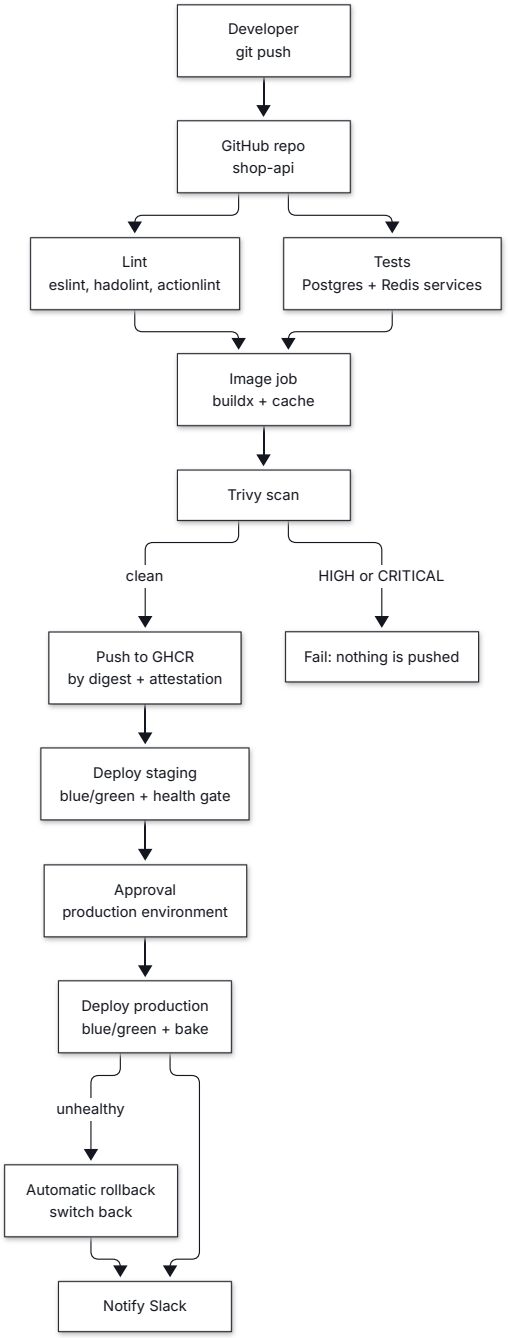


Figure 5-1

What runs on the server (one small Linux VM hosts both environments, separated by ports and directories):

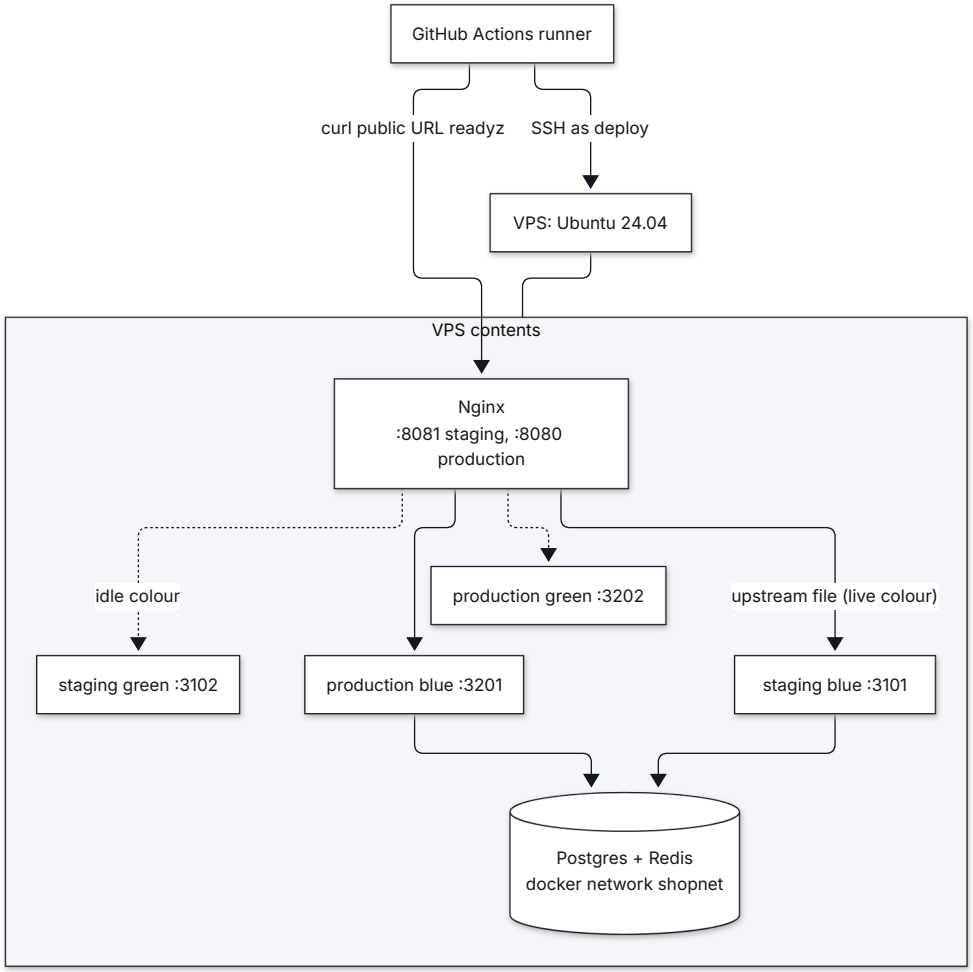


Figure 5-2

3 Prerequisites and cost

- **Accounts and tools:** a GitHub account (public repositories get free Actions minutes and free GHCR storage; private repositories use your plan's quota), `git`, `gh` (GitHub CLI, logged in), `ssh`, `jq`, Docker locally, Node.js 22.
- **A Linux server** reachable over SSH on a public IP: a small VPS (Ubuntu 24.04, 1 vCPU / 1 to 2 GB RAM, typically a few US dollars per month at most providers), a free-tier cloud VM, or a local VM with a bridged network if you only use your own runner or tunnel. The

capstone uses `203.0.113.10` as the illustrative address (RFC 5737; replace it).

- **Slack (optional):** an incoming webhook URL. Without it, the notify job prints to the run summary only.
- **Cost warning:** the VPS is the only cost. Destroy it in the teardown step. Do not leave a server with a public SSH port and open ports 8080/8081 running unattended. Set a billing alert with your provider.
- **Knowledge:** you should have completed topics L5.1 to L5.12 (or have them open), plus Docker §4.7 and Git and GitHub §3.11.
- **Time:** about 12 hours, spread over a few sessions. Milestones are independent enough to stop after each.

4 Build it step by step

Replace `<you>` with your GitHub user and `203.0.113.10` with your server's address throughout.

Milestone 0 — The application contract (≈1 h)

The pipeline assumes `shop-api` provides the following. If your sample app lacks them, add them now (you wrote most of this in earlier topics).

Requirement	Detail	Topic
`npm run lint` and `npm test`	lint exits non-zero on problems; tests use `DATABASE_URL`, `REDIS_URL`, `SESSION_SECRET` from the environment	L5.3
`GET /livez`	200 if the process is alive; no dependency checks	L5.8
`GET /readyz`	`{"status":"ok","version":"<APP_VERSION>"}` with 200 when ready, 503 while starting or shutting down	L5.8
Graceful `SIGTERM`	fail readiness, drain, close server, exit 0	L5.8
Fail-fast config	exit with a clear message if a required variable is missing; no secrets in the image	L5.5
`Dockerfile`	multi-stage, runs as non-root, `CMD ["node","src/server.js"]`, listens on `$PORT` (3000)	L5.4
Backwards-compatible migrations	optional `npm run migrate` that applies only expand-phase changes	L5.9

Create the repository and push it:

```bash
gh repo create shop-api --public --source=. --remote=origin --push
```

Milestone 1 — Prepare the server (≈1.5 h)

SSH to the server as root (or a sudo user) and run the following once. It installs Docker and Nginx, creates a restricted `deploy` user, a Docker network for the app, PostgreSQL and Redis containers, and an Nginx front door whose "live colour" is controlled by one small file per environment.

BASH

```
apt-get update && apt-get install -y nginx docker.io jq ufw
adduser --disabled-password --gecos "" deploy
usermod -aG docker deploy

# the deploy user may only validate and reload nginx, nothing else
echo 'deploy ALL=(root) NOPASSWD: /usr/sbin/nginx -t, /usr/bin/systemctl reload nginx' >
  /etc/sudoers.d/deploy-nginx
chmod 440 /etc/sudoers.d/deploy-nginx && visudo -cf /etc/sudoers.d/deploy-nginx

# upstream files: the pipeline rewrites these to switch colour (blue=3×01, green=3×02)
mkdir -p /etc/nginx/shop-api && chown deploy: /etc/nginx/shop-api
printf 'upstream shop_api_staging { server 127.0.0.1:3101; }\n'    > /etc/nginx/shop-
  api/staging-upstream.conf
printf 'upstream shop_api_production { server 127.0.0.1:3201; }\n' > /etc/nginx/shop-
  api/production-upstream.conf
chown deploy: /etc/nginx/shop-api/*.conf

cat > /etc/nginx/conf.d/shop-api.conf <<'EOF'
include /etc/nginx/shop-api/*.conf;

server {
  listen 8081;                      # staging
  location / {
    proxy_pass http://shop_api_staging;
    proxy_next_upstream error timeout http_502 http_503;
    proxy_set_header Host $host;
  }
}
server {
  listen 8080;                      # production
  location / {
    proxy_pass http://shop_api_production;
    proxy_next_upstream error timeout http_502 http_503;
    proxy_set_header Host $host;
  }
}
EOF
nginx -t && systemctl reload nginx

# firewall: SSH plus the two demo ports (restrict 8080/8081 to your IP if you can)
ufw allow 22/tcp && ufw allow 8080/tcp && ufw allow 8081/tcp && ufw --force enable

# shared network, database and cache (demo credentials: replace the passwords)
```

```
docker network create shopnet
docker run -d --name db --network shopnet --restart unless-stopped \
  -e POSTGRES_PASSWORD=<DB_PASSWORD> -v shop_pgdata:/var/lib/postgresql/data postgres:16
docker run -d --name cache --network shopnet --restart unless-stopped redis:7
sleep 5
docker exec db psql -U postgres -c "create database shop_staging" -c "create database
  shop_production"

# per-environment application settings (runtime configuration; never committed, never in
  the image)
for env in staging production; do
  install -d -o deploy -g deploy /home/deploy/shop-api-$env
  cat > /home/deploy/shop-api-$env/app.env <<EOF
DATABASE_URL=postgres://postgres:<DB_PASSWORD>@db:5432/shop_$env
REDIS_URL=redis://cache:6379
SESSION_SECRET=<RANDOM_STRING_$env>
LOG_LEVEL=$( [ $env = staging ] && echo debug || echo info )
EOF
  chown deploy: /home/deploy/shop-api-$env/app.env && chmod 600 /home/deploy/shop-
  api-$env/app.env
done
```

Create an SSH key for the pipeline on **your laptop** and authorise it on the server:

BASH

```
ssh-keygen -t ed25519 -N '' -C shop-api-pipeline -f deploy_key
ssh root@203.0.113.10 "install -d -m 700 -o deploy -g deploy /home/deploy/.ssh"
cat deploy_key.pub | ssh root@203.0.113.10 "cat >> /home/deploy/.ssh/authorized_keys &&
  chown deploy: /home/deploy/.ssh/authorized_keys && chmod 600
  /home/deploy/.ssh/authorized_keys"
ssh -i deploy_key deploy@203.0.113.10 'docker ps && sudo -n /usr/sbin/nginx -t'    #
  should list db and cache, and nginx syntax is ok
```

Milestone 2 — GitHub environments, variables and secrets (≈45 min)

Per-environment configuration lives in **Environments** (L5.5). The same workflow runs for both; only these values differ.

BASH

```
REPO=<you>/shop-api
gh api -X PUT repos/$REPO/environments/staging
gh api -X PUT repos/$REPO/environments/production     # then add Required reviewers in
  Settings > Environments

# shared (repository) values
gh variable set SSH_KNOWN_HOSTS --repo $REPO --body "$(ssh-keyscan -t ed25519 203.0.113.10
  2>/dev/null)"

for envname in staging production; do
  if [ $envname = staging ]; then P=1; PORT=8081; else P=2; PORT=8080; fi
```

```
  gh variable set DEPLOY_HOST    --repo $REPO --env $envname --body 203.0.113.10
  gh variable set PUBLIC_URL     --repo $REPO --env $envname --body
  "http://203.0.113.10:$PORT"
  gh variable set APP_NAME       --repo $REPO --env $envname --body "shop-api-$envname"
  gh variable set APP_DIR        --repo $REPO --env $envname --body "/home/deploy/shop-
  api-$envname"
  gh variable set BLUE_PORT      --repo $REPO --env $envname --body "3${P}01"
  gh variable set GREEN_PORT     --repo $REPO --env $envname --body "3${P}02"
  gh variable set PUBLIC_PORT    --repo $REPO --env $envname --body "$PORT"
  gh variable set UPSTREAM_NAME  --repo $REPO --env $envname --body "shop_api_$envname"
  gh variable set UPSTREAM_FILE  --repo $REPO --env $envname --body "/etc/nginx/shop-
  api/$envname-upstream.conf"
  gh secret set DEPLOY_SSH_KEY   --repo $REPO --env $envname < deploy_key
done
# optional
gh secret set SLACK_WEBHOOK_URL --repo $REPO --body '<YOUR_SLACK_WEBHOOK_URL>'
# optional: run migrations before the switch, e.g. gh variable set MIGRATE_CMD --env
  staging --body 'npm run migrate'
rm deploy_key deploy_key.pub    # the private key now lives only in GitHub's encrypted
  secrets (never commit it)
```

In **Settings, Environments, production** add yourself as a required reviewer and restrict deployment branches to `main`. In **Settings, Branches** protect `main` and require the checks `lint` and `test` (Protected Branches §3.8).

Milestone 3 — The pipeline: lint, test, build, scan, push (≈2.5 h)

Save as `.github/workflows/pipeline.yml`. It is the whole pipeline from the push to the call of the deploy workflow and the Slack notification. Pull requests run `lint`, `test` and the build-and-scan (no push, no deploy); pushes to `main` run everything.

YAML

```yaml
name: pipeline

on:
  push:
    branches: [main]
  pull_request:

concurrency:
  group: pipeline-${{ github.ref }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

permissions:
  contents: read

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}
```

```yaml
jobs:
  lint:
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm
      - run: npm ci
      - run: npm run lint
      - name: Lint the Dockerfile
        uses: hadolint/hadolint-action@v3.1.0
        with:
          dockerfile: Dockerfile
      - name: Lint the workflows
        run: docker run --rm -v "$PWD:/repo" -w /repo rhysd/actionlint:1.7.7 -color

  test:
    runs-on: ubuntu-24.04
    timeout-minutes: 15
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: ci-only-password
          POSTGRES_DB: shop_test
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U postgres"
          --health-interval 5s --health-timeout 5s --health-retries 10
      redis:
        image: redis:7
        ports: ["6379:6379"]
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 5s --health-timeout 5s --health-retries 10
    env:
      DATABASE_URL: postgres://postgres:ci-only-password@localhost:5432/shop_test
      REDIS_URL: redis://localhost:6379
      SESSION_SECRET: ci-only-not-a-secret
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm
      - run: npm ci
      - run: npm test

  image:
    needs: [lint, test]
```

```
    runs-on: ubuntu-24.04
    timeout-minutes: 25
    permissions:
      contents: read
      packages: write
      id-token: write
      attestations: write
    outputs:
      image: ${{ steps.ref.outputs.image }}
      version: ${{ github.sha }}
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3

      - name: Build for scanning (not pushed)
        uses: docker/build-push-action@v6
        with:
          context: .
          load: true
          tags: shop-api:scan
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - name: Security scan (fail on fixable HIGH or CRITICAL)
        uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: shop-api:scan
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          exit-code: "1"

      - name: Log in to GHCR
        if: github.event_name == 'push'
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Compute tags and labels
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
          flavor: latest=false
          tags: |
            type=sha,format=long,prefix=sha-
            type=ref,event=branch

      - name: Push image
        id: push
        if: github.event_name == 'push'
        uses: docker/build-push-action@v6
```

```
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
          provenance: mode=max
          sbom: true

      - name: Attest build provenance
        if: github.event_name == 'push'
        uses: actions/attest-build-provenance@v2
        with:
          subject-name: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
          subject-digest: ${{ steps.push.outputs.digest }}
          push-to-registry: true

      - name: Export the immutable image reference
        id: ref
        if: github.event_name == 'push'
        env:
          DIGEST: ${{ steps.push.outputs.digest }}
        run: |
          IMAGE="${REGISTRY}/${IMAGE_NAME,,}@${DIGEST}"
          echo "image=$IMAGE" >> "$GITHUB_OUTPUT"
          echo "### Image" >> "$GITHUB_STEP_SUMMARY"
          echo "\`$IMAGE\`" >> "$GITHUB_STEP_SUMMARY"

  staging:
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    needs: image
    uses: ./.github/workflows/deploy.yml
    with:
      environment: staging
      image: ${{ needs.image.outputs.image }}
      version: ${{ needs.image.outputs.version }}
    secrets: inherit
    permissions:
      contents: read
      packages: read
      attestations: read

  production:
    needs: [image, staging]
    uses: ./.github/workflows/deploy.yml
    with:
      environment: production
      image: ${{ needs.image.outputs.image }}
      version: ${{ needs.image.outputs.version }}
    secrets: inherit
    permissions:
      contents: read
```

```
      packages: read
      attestations: read

  notify:
    if: always() && github.event_name == 'push'
    needs: [lint, test, image, staging, production]
    runs-on: ubuntu-24.04
    timeout-minutes: 5
    steps:
      - name: Post the result to Slack
        env:
          SLACK_WEBHOOK_URL: ${{ secrets.SLACK_WEBHOOK_URL }}
          LINT: ${{ needs.lint.result }}
          TEST: ${{ needs.test.result }}
          IMAGE: ${{ needs.image.result }}
          STAGING: ${{ needs.staging.result }}
          PRODUCTION: ${{ needs.production.result }}
        run: |
          MSG="shop-api ${GITHUB_SHA::7} by ${GITHUB_ACTOR}: lint=$LINT test=$TEST
image=$IMAGE staging=$STAGING production=$PRODUCTION
${GITHUB_SERVER_URL}/${GITHUB_REPOSITORY}/actions/runs/${GITHUB_RUN_ID}"
          echo "$MSG" >> "$GITHUB_STEP_SUMMARY"
          if [ -z "${SLACK_WEBHOOK_URL:-}" ]; then
            echo "SLACK_WEBHOOK_URL not set: skipping Slack"
            exit 0
          fi
          curl -fsS -X POST -H 'Content-type: application/json' \
            --data "$(jq -n --arg text "$MSG" '{text: $text}')" "$SLACK_WEBHOOK_URL"
```

Things to notice (each maps to a topic):

- `lint` and `test` run **in parallel**; `image` waits for both (fail fast, L5.1). `concurrency` cancels superseded PR runs but queues `main` runs (L5.2).
- The image is **built once**, scanned **before** it is pushed, and only then pushed (re-using the cache) with provenance and an SBOM; the output is the immutable `repo@sha256:...` reference (L5.4, L5.6).
- Both deploy jobs receive the **same digest**; only the environment differs (L5.6).
- `permissions` are minimal at the top and widened only on the image job; the registry login uses `GITHUB_TOKEN`, with no stored password (L5.5).

Milestone 4 — The deploy script: blue/green, health gate, bake, rollback (≈2 h)

Save as `scripts/deploy.sh`. It runs **on the server** (the workflow streams it over SSH, so the version in Git is always the version that runs). It deploys to the idle colour, gates on `/readyz` reporting the new version, switches Nginx, bakes, and switches back automatically if the

public endpoint misbehaves. The control flow of every path (first deploy, upgrade, never-healthy, failing during bake, instant and slow rollback) was exercised against stubbed `docker`, `curl` and `nginx` commands; on a real server expect timing differences.

BASH

```
#!/usr/bin/env bash
# Blue/green deploy with health gate, bake time and automatic rollback.
#   deploy.sh deploy <image-ref> <version>     deploy a new version to the idle colour
#   deploy.sh rollback                         switch back to the previous colour (or
  redeploy previous)
# Required env: APP_NAME APP_DIR BLUE_PORT GREEN_PORT PUBLIC_PORT UPSTREAM_NAME
  UPSTREAM_FILE
# Optional env: MIGRATE_CMD (run once in a throwaway container before the switch),
  BAKE_SECONDS, DOCKER_NETWORK
# Exit codes: 0 ok, 1 failed before switching, 2 failed after switching and rolled back, 3
  rollback failed
set -euo pipefail

CMD="${1:?usage: deploy.sh deploy <image> <version> | rollback}"
: "${APP_NAME:?}" "${APP_DIR:?}" "${BLUE_PORT:?}" "${GREEN_PORT:?}" "${PUBLIC_PORT:?}"
  "${UPSTREAM_NAME:?}" "${UPSTREAM_FILE:?}"
BAKE_SECONDS="${BAKE_SECONDS:-30}"
NETWORK="${DOCKER_NETWORK:-shopnet}"      # user-defined network that also holds the db
  and cache containers
mkdir -p "$APP_DIR"
cd "$APP_DIR"

live="$(cat live.color 2>/dev/null || echo none)"
port_of() { if [ "$1" = blue ]; then echo "$BLUE_PORT"; else echo "$GREEN_PORT"; fi; }
other()   { if [ "$1" = blue ]; then echo green; else echo blue; fi; }

serving() {   # serving <port> <version>: readiness says ok AND reports this version
  curl -fsS --max-time 3 "http://127.0.0.1:$1/readyz" | jq -e --arg v "$2" '.status ==
  "ok" and .version == $v' >/dev/null 2>&1
}

wait_ready() {   # wait_ready <port> <version>: 5 consecutive good answers within ~2
  minutes
  local ok=0
  for _ in $(seq 1 60); do
    if serving "$1" "$2"; then ok=$((ok+1)); else ok=0; fi
    [ "$ok" -ge 5 ] && return 0
    sleep 2
  done
  return 1
}

switch_to() {   # switch_to <colour>: atomic upstream swap + validated graceful reload
  printf 'upstream %s { server 127.0.0.1:%s; }\n' "$UPSTREAM_NAME" "$(port_of "$1")" >
  "$UPSTREAM_FILE.new"
  mv "$UPSTREAM_FILE.new" "$UPSTREAM_FILE"
```

```
  sudo -n /usr/sbin/nginx -t
  sudo -n /usr/bin/systemctl reload nginx
  echo "$1" > live.color
  echo "traffic now goes to $1 ($(port_of "$1"))"
}

start_colour() {   # start_colour <colour> <image> <version>
  local name="$APP_NAME-$1" port
  port="$(port_of "$1")"
  docker rm -f "$name" >/dev/null 2>&1 || true
  docker pull "$2"
  docker run -d --name "$name" --network "$NETWORK" --restart unless-stopped --stop-
  timeout 30 \
    --env-file "$APP_DIR/app.env" -e APP_VERSION="$3" -e PORT=3000 \
    -p "127.0.0.1:$port:3000" \
    --health-cmd "wget -qO- http://127.0.0.1:3000/livez || exit 1" --health-interval 5s --
  health-retries 5 \
    "$2" >/dev/null
}

bake() {   # bake <version>: watch the PUBLIC endpoint for BAKE_SECONDS, tolerate at most
  2 failures
  local fails=0 end=$((SECONDS+BAKE_SECONDS))
  while [ "$SECONDS" -lt "$end" ]; do
    serving "$PUBLIC_PORT" "$1" || fails=$((fails+1))
    [ "$fails" -gt 2 ] && return 1
    sleep 2
  done
  return 0
}

do_deploy() {   # do_deploy <image> <version>
  local image="$1" version="$2" idle
  idle="$(other "$live")"; [ "$live" = none ] && idle=blue
  echo "live=$live idle=$idle version=$version"

  if [ -n "${MIGRATE_CMD:-}" ]; then
    echo "running migrations (expand only): $MIGRATE_CMD"
    docker run --rm --network "$NETWORK" --env-file "$APP_DIR/app.env" "$image" sh -c
  "$MIGRATE_CMD"
  fi

  start_colour "$idle" "$image" "$version"
  if ! wait_ready "$(port_of "$idle")" "$version"; then
    echo "DEPLOY FAILED: $idle never became healthy on $version; NOT switching" >&2
    docker logs --tail 50 "$APP_NAME-$idle" >&2 || true
    exit 1
  fi

  switch_to "$idle"
  if ! bake "$version"; then
    echo "BAKE FAILED: switching back to $live" >&2
    if [ "$live" ≠ none ]; then
```

```
      switch_to "$live"
      exit 2
    fi
    exit 1
  fi

  [ -f current.version ] && cp current.version previous.version && cp current.image
  previous.image
  echo "$version" > current.version
  echo "$image" > current.image
  echo "DEPLOY OK: $version live on $idle ($live kept warm for rollback)"
}

do_rollback() {
  [ "$live" =≠ none ] || { echo "nothing to roll back" >&2; exit 3; }
  local target prev_version
  target="$(other "$live")"
  prev_version="$(cat previous.version 2>/dev/null || true)"
  if [ -n "$prev_version" ] && serving "$(port_of "$target")" "$prev_version"; then
    echo "instant rollback: $target is still running $prev_version"
    switch_to "$target"
    for f in version image; do                        # swap the current/previous records
      mv "current.$f" "tmp.$f"; mv "previous.$f" "current.$f"; mv "tmp.$f" "previous.$f"
    done
  elif [ -f previous.image ]; then
    echo "previous colour is gone: redeploying $(cat previous.image)"
    do_deploy "$(cat previous.image)" "$prev_version"
  else
    echo "ROLLBACK IMPOSSIBLE: no previous release recorded" >&2
    exit 3
  fi
  echo "ROLLED BACK to $prev_version"
}

case "$CMD" in
  deploy)   do_deploy "${2:?image}" "${3:?version}" ;;
  rollback) do_rollback ;;
  *) echo "unknown command $CMD" >&2; exit 1 ;;
esac
```

Exit codes carry the story: `0` success, `1` failed before any switch (live version untouched), `2` failed after switching and was rolled back, `3` the rollback itself failed (page a human). The previous colour keeps running after a successful deploy, so a manual rollback is instant (L5.7, L5.11). Migrations, when configured through `MIGRATE_CMD`, run once in a throwaway container before the switch and must be expand-only (L5.9).

Milestone 5 — The reusable deploy workflow (≈1.5 h)

Save as `.github/workflows/deploy.yml`. It is called by the pipeline for staging and production, and can also be run by hand with an image reference. It verifies the image's **attestation** (only images built by this repository's workflow are deployed), runs the script, then checks the result **from outside**, through the public URL, as a user would.

YAML

```yaml
name: deploy

on:
  workflow_call:
    inputs:
      environment:
        type: string
        required: true
      image:
        description: Immutable image reference (registry/repo@sha256:...)
        type: string
        required: true
      version:
        description: Version reported by /readyz (the commit SHA)
        type: string
        required: true
  workflow_dispatch:
    inputs:
      environment:
        type: choice
        options: [staging, production]
        required: true
      image:
        description: Immutable image reference (registry/repo@sha256:...)
        required: true
      version:
        description: Commit SHA that built the image
        required: true

permissions:
  contents: read
  packages: read
  attestations: read

jobs:
  deploy:
    runs-on: ubuntu-24.04
    timeout-minutes: 20
    environment: ${{ inputs.environment }}
    concurrency:
      group: deploy-${{ inputs.environment }}
      cancel-in-progress: false
    env:
```

```
      GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
      IMAGE: ${{ inputs.image }}
      VERSION: ${{ inputs.version }}
      HOST: ${{ vars.DEPLOY_HOST }}
      PUBLIC_URL: ${{ vars.PUBLIC_URL }}
    steps:
      - uses: actions/checkout@v4

      - name: Verify provenance (must be built by this repository's workflow)
        run: gh attestation verify "oci://$IMAGE" --repo "$GITHUB_REPOSITORY"

      - name: Configure SSH
        env:
          SSH_KEY: ${{ secrets.DEPLOY_SSH_KEY }}
          KNOWN_HOSTS: ${{ vars.SSH_KNOWN_HOSTS }}
        run: |
          install -m 700 -d ~/.ssh
          printf '%s\n' "$SSH_KEY" > ~/.ssh/id_ed25519
          chmod 600 ~/.ssh/id_ed25519
          printf '%s\n' "$KNOWN_HOSTS" > ~/.ssh/known_hosts

      - name: Deploy (migrate, blue/green, health gate, bake, auto rollback)
        id: deploy
        env:
          APP_NAME: ${{ vars.APP_NAME }}
          APP_DIR: ${{ vars.APP_DIR }}
          BLUE_PORT: ${{ vars.BLUE_PORT }}
          GREEN_PORT: ${{ vars.GREEN_PORT }}
          PUBLIC_PORT: ${{ vars.PUBLIC_PORT }}
          UPSTREAM_NAME: ${{ vars.UPSTREAM_NAME }}
          UPSTREAM_FILE: ${{ vars.UPSTREAM_FILE }}
          MIGRATE_CMD: ${{ vars.MIGRATE_CMD }}
        run: |
          set +e
          ssh "deploy@$HOST" \
            "APP_NAME='$APP_NAME' APP_DIR='$APP_DIR' BLUE_PORT='$BLUE_PORT'
GREEN_PORT='$GREEN_PORT' PUBLIC_PORT='$PUBLIC_PORT' UPSTREAM_NAME='$UPSTREAM_NAME'
UPSTREAM_FILE='$UPSTREAM_FILE' MIGRATE_CMD='$MIGRATE_CMD' bash -s -- deploy '$IMAGE'
'$VERSION'" \
            < scripts/deploy.sh
          rc=$?
          echo "rc=$rc" >> "$GITHUB_OUTPUT"
          exit $rc

      - name: External health check (from the runner, through the public URL)
        run: |
          ok=0
          for i in $(seq 1 30); do
            if curl -fsS --max-time 5 "$PUBLIC_URL/readyz" | jq -e --arg v "$VERSION"
'.status == "ok" and .version == $v' >/dev/null; then
              ok=$((ok+1))
            else
              ok=0
```

```
            fi
            [ "$ok" -ge 5 ] && { echo "healthy: $ok consecutive successes"; exit 0; }
            sleep 2
          done
          echo "external health check FAILED" >&2
          exit 1

      - name: Summary
        if: always()
        env:
          RC: ${{ steps.deploy.outputs.rc }}
          ENVIRONMENT: ${{ inputs.environment }}
        run: |
          case "${RC:-unknown}" in
            0) R="deployed" ;;
            1) R="FAILED before switching (live version untouched)" ;;
            2) R="FAILED after switching and was ROLLED BACK automatically" ;;
            3) R="FAILED and rollback failed: PAGE A HUMAN" ;;
            *) R="did not complete (rc=${RC:-unknown})" ;;
          esac
          echo "### $ENVIRONMENT: $R" >> "$GITHUB_STEP_SUMMARY"
          echo "image: \`$IMAGE\`" >> "$GITHUB_STEP_SUMMARY"
```

Milestone 6 — One-click rollback (≈45 min)

Save as `.github/workflows/rollback.yml`. It reuses the same script: it switches to the previous colour if that colour is still running the previous release (seconds), otherwise it redeploys the previous image.

YAML

```yaml
name: rollback

on:
  workflow_dispatch:
    inputs:
      environment:
        type: choice
        options: [staging, production]
        required: true

permissions:
  contents: read

jobs:
  rollback:
    runs-on: ubuntu-24.04
    timeout-minutes: 15
    environment: ${{ inputs.environment }}
    concurrency:
      group: deploy-${{ inputs.environment }}
      cancel-in-progress: false
```

```
    steps:
      - uses: actions/checkout@v4

      - name: Configure SSH
        env:
          SSH_KEY: ${{ secrets.DEPLOY_SSH_KEY }}
          KNOWN_HOSTS: ${{ vars.SSH_KNOWN_HOSTS }}
        run: |
          install -m 700 -d ~/.ssh
          printf '%s\n' "$SSH_KEY" > ~/.ssh/id_ed25519
          chmod 600 ~/.ssh/id_ed25519
          printf '%s\n' "$KNOWN_HOSTS" > ~/.ssh/known_hosts

      - name: Roll back to the previous release
        env:
          HOST: ${{ vars.DEPLOY_HOST }}
          APP_NAME: ${{ vars.APP_NAME }}
          APP_DIR: ${{ vars.APP_DIR }}
          BLUE_PORT: ${{ vars.BLUE_PORT }}
          GREEN_PORT: ${{ vars.GREEN_PORT }}
          PUBLIC_PORT: ${{ vars.PUBLIC_PORT }}
          UPSTREAM_NAME: ${{ vars.UPSTREAM_NAME }}
          UPSTREAM_FILE: ${{ vars.UPSTREAM_FILE }}
        run: |
          ssh "deploy@$HOST" \
            "APP_NAME='$APP_NAME' APP_DIR='$APP_DIR' BLUE_PORT='$BLUE_PORT'
    GREEN_PORT='$GREEN_PORT' PUBLIC_PORT='$PUBLIC_PORT' UPSTREAM_NAME='$UPSTREAM_NAME'
    UPSTREAM_FILE='$UPSTREAM_FILE' bash -s -- rollback" \
            < scripts/deploy.sh

      - name: Show what is live now
        env:
          PUBLIC_URL: ${{ vars.PUBLIC_URL }}
        run: |
          curl -fsS "$PUBLIC_URL/readyz" | tee /dev/stderr | jq -e '.status == "ok"'
    >/dev/null
          echo "rolled back; live version: $(curl -fsS "$PUBLIC_URL/readyz" | jq -r
    .version)" >> "$GITHUB_STEP_SUMMARY"
```

Milestone 7 — First run, then validate (≈1 h)

BASH

```
git add . && git commit -m "ci: add pipeline" && git push origin main
gh run watch                                    # follow the pipeline
curl -s http://203.0.113.10:8081/readyz         # staging: {"status":"ok","version":"
  <commit sha>"}
```

After the staging job passes, the `production` job pauses for approval: approve it in the Actions UI, then `curl -s http://203.0.113.10:8080/readyz`. Make the GHCR package public (package settings) or run `docker login ghcr.io` once as `deploy` if the server cannot pull the image. Lint your workflows locally any time with `docker run --rm -v "$PWD:/repo" -w /repo rhysd/actionlint:1.7.7`.

Milestone 8 — Compare the other strategies (≈2 h, optional but recommended)

Blue/green is what you built. To compare with the other two strategies from this level without extra cost, use a local kind cluster: deploy `shop-api` as a rolling update with `maxSurge: 1, maxUnavailable: 0` (L5.10), then install Argo Rollouts and run a canary with `setWeight: 20`, an analysis step, and a promotion (L5.12). Document in a table: rollback speed, extra cost, exposure of a bad release, operational complexity.

5 Tasks

- [ ] **T1 — Bootstrap the server and prove it is safe** *(≈90 min, intermediate; milestones 1 and 2)*
  - **Goal:** a hardened deploy target and complete per-environment configuration. Exercises L5.5 and L5.8.
  - **Steps:**
    1. Run the bootstrap commands; create the SSH key and authorise it; store it only as environment secrets.
    2. Set all variables for `staging` and `production` and enable required reviewers on production.
    3. As the `deploy` user, verify you can run `docker ps` and `sudo -n /usr/sbin/nginx -t` but not `sudo -n bash`.
  - **Done when:** `ssh -i key deploy@host 'sudo -n bash -c id'` is refused, `gh variable list --env production` shows all values, and no secret exists in the repository files.
- [ ] **T2 — Green CI with real services** *(≈90 min, intermediate; milestone 3)*
  - **Goal:** lint and test jobs that run in parallel and gate everything. Exercises L5.1, L5.2, L5.3.
  - **Steps:**
    1. Add `pipeline.yml` (lint and test jobs first) and open a pull request.
    2. Make a test fail on purpose and a lint rule fail on purpose, in separate commits; observe that `image` never starts.
    3. Add the required status checks to branch protection.

- **Done when:** the PR shows `lint` and `test` side by side, a red commit blocks merging, and the run page shows `image` as skipped.

- [ ] **T3 — Image, cache, scan, provenance** *(≈2 h, intermediate; milestone 3)*
  - **Goal:** an image that is scanned before it is pushed and traceable to a commit. Exercises L5.4 and L5.6.
  - **Steps:**
    1. Merge to `main` and read the image job summary for the digest.
    2. Re-run to see the cache hit and note the build time difference.
    3. Add a deliberately vulnerable base image (an old `node` tag) in a branch and confirm the scan fails the job; revert.
    4. Run `gh attestation verify oci://<image-ref> --repo <you>/shop-api`.
  - **Done when:** the registry has `sha-<commit>` and the digest verifies, the second build is clearly faster, and the vulnerable branch never pushed an image.

- [ ] **T4 — Deploy to staging and production with a health gate** *(≈2 h, advanced; milestones 4 to 7)*
  - **Goal:** the first complete push-to-production run. Exercises L5.5, L5.8, L5.9, L5.11.
  - **Steps:**
    1. Add `scripts/deploy.sh`, `deploy.yml` and `rollback.yml`; push to `main`.
    2. Watch staging go green, approve production, and then deploy a second commit so that colours swap.
    3. While deploying the second commit, run `while sleep 0.5; do curl -s -o /dev/null -w '%{http_code}\n' http://203.0.113.10:8081/readyz; done | sort | uniq -c` in another terminal.
    4. Make a harmless expand-only migration (add a nullable column) and run it through `MIGRATE_CMD` on staging.
  - **Done when:** both environments serve the latest commit's SHA at `/readyz`, the second deployment changed the live colour (`cat live.color` on the server) and the status-code count shows only `200`.

- [ ] **T5 — Break it: bad releases and the rollback paths** *(≈2 h, advanced; milestones 4 to 6)*
  - **Goal:** prove that each failure ends safely. Exercises L5.7 and L5.8.
  - **Steps:**
    1. Add drill switches to the app, for example `FAIL_READY=1` (readiness always 500) and `FAIL_AFTER=20` (healthy for 20 seconds, then 500), set through `app.env` on staging. With `FAIL_READY=1`, deploy: expect exit code 1 and the live version unchanged.

2. With `FAIL_AFTER=20`, deploy again (healthy at start, failing during the bake): expect exit code 2 and an automatic switch back.
3. Deploy a good version, then run the `rollback` workflow and measure how long traffic takes to move back.
4. Break the rollback target (stop the idle container with `docker stop`) and roll back again; observe the redeploy path.

- **Done when:** you have the run summaries for all four cases, each stating the right outcome, and `/readyz` always showed a healthy version during and after.

- [ ] **T6 — Expand/contract migration drill** *(≈2 h, advanced; milestone 4)*
  - **Goal:** change the schema without an outage or an unrecoverable rollback. Exercises L5.9.
  - **Steps:**
    1. Plan the three-release change to turn `orders.total` into `orders.total_cents`: expand with a trigger, backfill and dual-write release, then contract.
    2. Ship the expand release through the pipeline; roll back to the previous release and confirm it still works on the new schema.
    3. Ship the migrate release; ship the contract release only after the rollback window.
  - **Done when:** each release deployed and passed its health gate, a rollback after the expand release succeeded, and the contract release was the only one that could not be rolled back past.

- [ ] **T7 — Compare rolling, blue/green and canary** *(≈2 h, advanced; milestone 8)*
  - **Goal:** choose a strategy with evidence. Exercises L5.10, L5.11, L5.12.
  - **Steps:**
    1. Run a rolling update on kind with a loop of requests and count the codes.
    2. Run the local canary script from L5.12 with a healthy and a failing canary.
    3. Fill in a table comparing rollback time, cost, exposure of a bad version, and complexity for all three strategies using your measurements and this capstone's blue/green runs.
  - **Done when:** the table has measured values (not guesses) for at least rollback time and error count, with a recommendation for `shop-api`.

- [ ] **T8 — Prove it: rebuild the pipeline from a blank repository** *(≈3 h, advanced; all milestones)*
  - **Goal:** demonstrate you can design and operate this pipeline without the guide.
  - **Steps:**

1. In a new repository with only the application, write the workflows and the deploy script from memory and a clean server; use the guide only for syntax lookups.
2. Trigger one successful release and one bad release that rolls back automatically.
3. Present it in 10 minutes: the stage diagram, what each gate proves, where secrets live, how rollback works, and what you would change at ten times the traffic.

- **Done when:** the second release is rolled back with no human action, a reviewer can follow the run summaries, and you can answer why the image is built once, scanned before pushing and deployed by digest.

6 Verification checklist

- [ ] A push to `main` triggers the full pipeline; a pull request runs lint, test, build and scan but pushes and deploys nothing.
- [ ] `lint` and `test` run in parallel; a failing test or lint rule blocks the image job.
- [ ] The scan runs before the push and fails on fixable HIGH or CRITICAL findings.
- [ ] The registry contains `sha-<commit>`, the deploy used the `@sha256:` digest, and `gh attestation verify` succeeds for it.
- [ ] Staging and production run the **same digest**, built once; the only differences are environment variables and secrets.
- [ ] Production waits for a human approval; only `main` may deploy to it.
- [ ] No secret appears in the repository, in logs (check the run logs), or in the image (`docker history --no-trunc`).
- [ ] `curl http://<host>:<port>/readyz` returns the commit SHA of the latest release.
- [ ] A release that never becomes healthy leaves the live version untouched (exit code 1).
- [ ] A release that fails during the bake is rolled back automatically (exit code 2) and the Slack or summary message says so.
- [ ] The `rollback` workflow returns to the previous release in seconds when the previous colour is still running.
- [ ] A request loop during a normal deploy shows no non-2xx responses.
- [ ] Schema changes were released as expand, migrate, contract; a rollback after the expand release works.
- [ ] You can state the cost of the blue/green overlap and the retention policy for old images.

7 Teardown

1. **Destroy the server** at your provider (or delete the VM), and remove its firewall rules and any reserved IP. This stops the only recurring cost.
2. **Revoke access:** delete the `shop-api-pipeline` public key wherever you copied it, remove the repository secrets and variables (`gh secret delete DEPLOY_SSH_KEY --env staging`, and likewise for production and `SLACK_WEBHOOK_URL`), and revoke the Slack webhook.
3. **Delete the registry data:** GHCR package versions for `shop-api` (Packages, Package settings, Delete) if you no longer need them.
4. **Remove the environments:** `gh api -X DELETE repos/<you>/shop-api/environments/staging` and `.../production`.
5. **Clean local files:** `rm -f deploy_key deploy_key.pub`, remove kind clusters (`kind delete cluster`) and Docker leftovers (`docker system prune`).
6. Verify the provider billing page shows no running instances and no unattached volumes or IPs.

8 Stretch goals and what to learn next

- Add a **canary** stage: keep 90/10 weights in the Nginx upstream for 10 minutes with a Prometheus-based error-rate check before 100% (L5.12), or replace the VM with Kubernetes and Argo Rollouts.
- Add **preview environments** per pull request (a temporary container on another port, destroyed on PR close).
- Replace SSH deployment with **OIDC to a cloud account** (no stored keys) and deploy to ECS or EKS; build the infrastructure with Terraform.
- Move the desired state to Git and let a controller apply it (**GitOps**): the pipeline only builds and bumps an image digest in a config repository.
- Add DAST (OWASP ZAP) against staging, secret scanning in CI, and Dependabot for actions and dependencies (Level 11 §11.9).
- Add metrics and SLO-based gates to the bake step (Level 10 §10.2, Level 14 §14.4).

Next level: Level 6 — Cloud (AWS) §6.1 moves this pipeline onto managed infrastructure: EC2, IAM roles for OIDC, ALB for the blue/green switch, and Secrets Manager.